

0.23. Sans « x = » : la console range la position

Le 0.20 sans les « x = » : `deplace_x(x, ...)`; seul sur sa ligne, et x change quand même. Les nombres le montrent.

1. Ce qu'il faut comprendre

C'est le 0.20, avec un seul changement : les deux `x =` ont disparu.

Ce qui est nouveau ici : quand l'appel est **seul sur sa ligne** et que la position donnée est une **variable** (`x`), la console **range elle-même** la position d'arrivée dans cette variable. Le compilateur réécrit la ligne en `deplace_x(x, ...)` : c'est exactement le 0.20, la même cartouche, octet pour octet.

La preuve : les nombres affichent toujours `005` puis `000`. Personne n'a écrit `x =`, et pourtant `x` a changé.

C'est une exception, réservée à la console. La règle du C ne change pas : une fonction reçoit une copie. Seules les fonctions de la console (`deplace_x`, `deplace_y`, et celles qui viendront) ont droit à ce rangement. Pour **tes** fonctions, il faut toujours `x =` (le 0.33 le montre).

Une condition : la position doit être une **variable**. `deplace_x(3, 0, ALPHABET[0], 5)` fait bouger la lettre, mais il n'y a nulle part où ranger la colonne.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction `deplace\_x\(\)` — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1145 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
main()         69 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`,

Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans deplace_x() + AdresseCase (10 o)
nombre()	× 2	20 o sur place, dans main() + EcrireNombre (33 o)
effacer()	× 1	15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 2	6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)

Tes fonctions appelées :

deplace_x()	× 2	38 o sur place, dans main()
-------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <nombre> // écrit un nombre en chiffres
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // +5 : avance ; -5 : recule
```

3. UNE VARIABLE — X

```
uint8_t x = 0; // la colonne de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le changement : plus de « x = » devant deplace_x.
    //
    // deplace_x(x, 0, ALPHABET[0], pas);
    //
    // |
    // +-- x est une VARIABLE : la console y range toute seule la
    // colonne d'arrivée. Le compilateur réécrit la ligne en
    // x = deplace_x(x, 0, ALPHABET[0], pas);
    // C'est le 0.20, écrit plus court.
    deplace_x(x, 0, ALPHABET[0], pas); // x vaut 5 quand même...
    nombre(0, 2, x); // ...la preuve : 005

    deplace_x(x, 0, ALPHABET[0], -pas); // x vaut 0
    nombre(0, 2, x); // 000

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



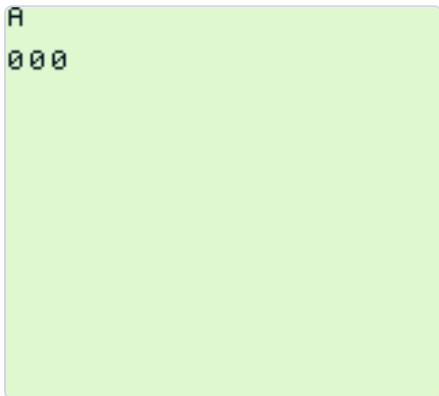
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Exactement le 0.20 : le A avance, 005 s'écrit ; il revient, 000 s'écrit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ sans « x = », x a reçu 5 : 005 affiché

✓ puis 0 : 000 affiché (x = 0)

✓ le A est revenu en colonne 0

Ensuite : **0.23.1. Sans « x = » : la console range la position — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++